
Oliver Krauss

Modern Code - AI-Assisted Coding

ModernCode | MTD.ba VZ SS25

Hagenberg 26.01.2026

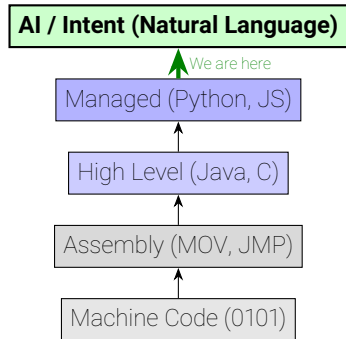
HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

The Paradigm Shift

- **History of Programming:** A journey of increasing abstraction.
- We started with electrical signals, then moved to binary, then mnemonics...
- Each step removed us further from the “metal” but allowed us to build more complex systems.
- **The New Layer:** AI is the next layer of abstraction.



From Imperative to Declarative

Traditional (Imperative)

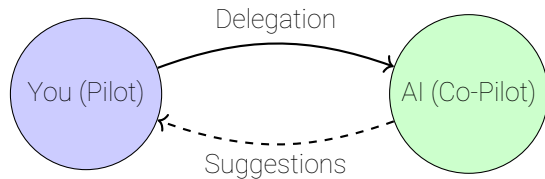
- You tell the computer *exactly* how to do something.
- "Loop from 1 to 10, add i to sum..."
- Requires deep syntax knowledge.

AI-Assisted (Declarative)

- You tell the computer *what result* you want.
- "Calculate the sum of the first 10 integers."
- Requires **Intent** and **Verification**.

The skill shifts from "Generating Syntax" to "Verifying Logic".

The Pilot Metaphor



Responsibility

Structure
Correctness
Business Logic

Speed

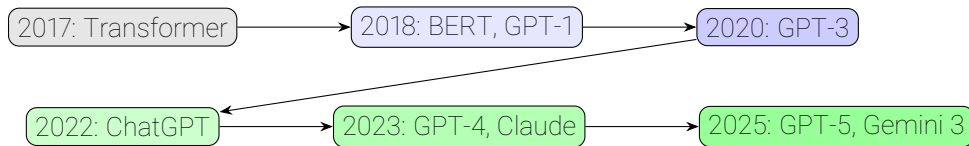
Boilerplate
Syntax
Drafts

- **You are the Pilot:** You decide the destination. You land the plane.
- **AI is the Co-Pilot:** It handles the routine, checks the maps, suggests routes.
- **Danger:** If the pilot falls asleep, the plane crashes.

Why LLMs Matter

- **Revolution in NLP:** LLMs have transformed how we interact with AI.
- **Human-like text generation:** Often indistinguishable from human writing.
- **Applications across industries:**
 - Software development (code generation, debugging)
 - Content creation and writing
 - Customer service (chatbots)
 - Education and research
 - Healthcare documentation
- **Foundation for AI Assistants:** ChatGPT, Claude, Copilot are all built on LLMs.

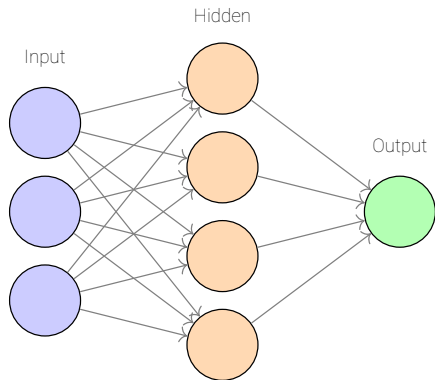
Evolution of Language Models



Scale matters. Larger models + more data = emergent capabilities.

What is a Neural Network? (Crash Course)

- A **Neural Network** is a system of interconnected "neurons" (mathematical functions).
- **Layers**: Input $\rightarrow$ Hidden Layers $\rightarrow$ Output.
- Each neuron receives inputs, multiplies by **weights**, sums, and applies an **activation function**.
- **Learning** = Adjusting weights to minimize prediction error.



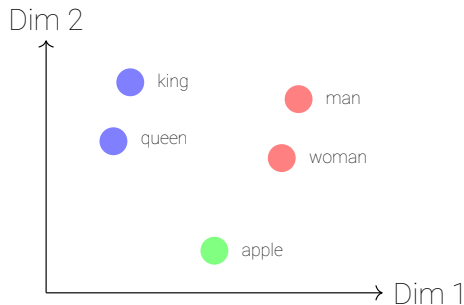
The Sequential Problem: Why Not RNNs?

- **RNNs (Recurrent Neural Networks)** process tokens *one at a time*.
- **Problem 1: No parallelization.** Training is slow.
- **Problem 2: Long-range dependencies.** Information must flow through many steps; early tokens are "forgotten".
- **Question:** Can we process all tokens *in parallel* while still capturing relationships?

Answer: Yes! The Transformer architecture.

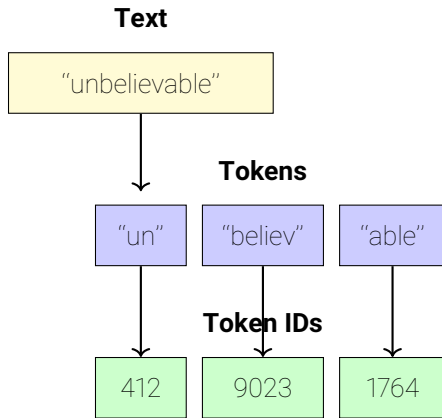
Word Embeddings: How Text Becomes Math

- Words are represented as **vectors** in a continuous space.
- **Similar words** have **similar vectors**.
- Famous example: **king** - **man** + **woman** = **queen**
- LLMs learn embeddings during training.



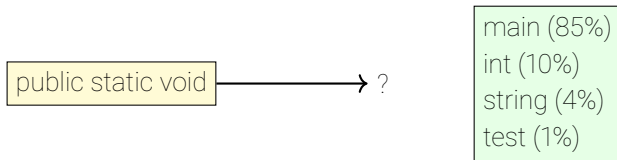
Tokenization

- LLMs don't read words. They read **Tokens** (sub-word units).
- "unbelievable" → "un", "believ", "able"
- Code is also tokenized: **System.out** → **System**, **.**, **out**



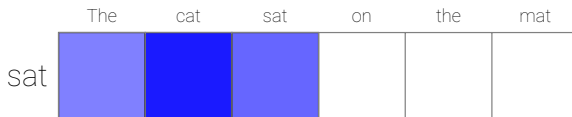
What is an LLM Really?

- **Not a Database:** It doesn't "know" facts; it stores *associations*.
- **Next Token Predictor:** It predicts the most likely next piece of text based on the statistical patterns of its training data.
- **Stochastic:** It is inherently random (probabilistic).



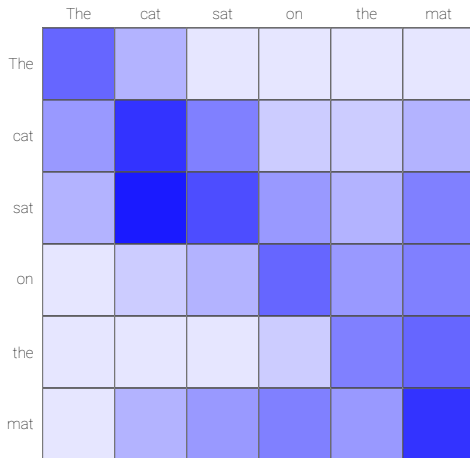
The Magic: Attention Mechanism

Attention allows the model to "focus" on relevant parts of the input.



When predicting what comes after "sat", the model "pays attention" strongly to **"cat"** (the subject). It learns grammar and context dynamically.

Attention: The Full Picture

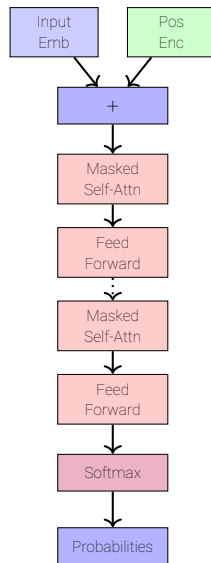


Darker = stronger attention. "sat" attends most to "cat" (its subject).

Architecture: The Transformer (Decoder-Only)

Modern LLMs (GPT, Claude) use a **Decoder-Only** Transformer architecture.

- **Input Embedding:** Text to Vector.
- **Causal Masking:** The model can only see *past* tokens, not future ones.
- **Self-Attention:** It determines which previous words are relevant to the current prediction.



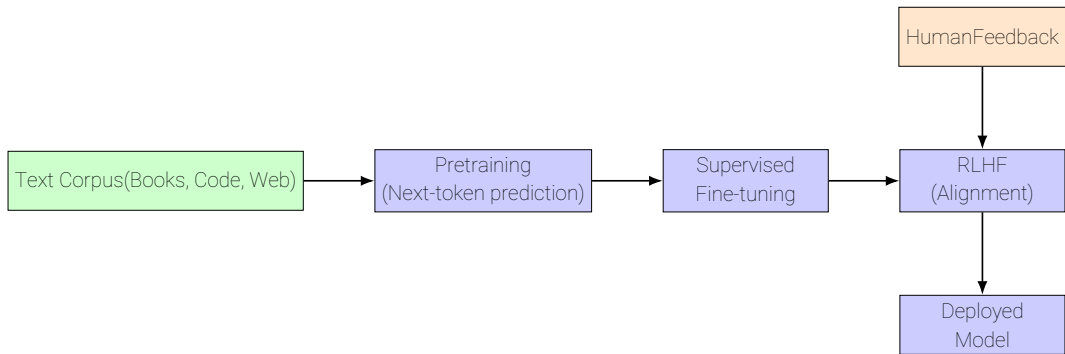
Sampling: Determinism vs Creativity

- The model outputs **probabilities** (e.g., int: 40%, String: 30%, void: 30%).
- **Temperature**: Controls how randomness is applied.
 - **Temp 0.0**: Always pick the top choice. Deterministic. (Good for Coding).
 - **Temp 1.0**: Pick randomly based on weights. Creative. (Good for Poetry).

Coding Tip

When generating code, you generally want **Low Temperature** to avoid syntax errors and "hallucinated" libraries.

How They Are Trained (The Pipeline)



Scaling Laws: Why Bigger = Smarter

- **Scaling Laws**: Performance improves predictably with:
 - More **parameters** (model size)
 - More **data** (training corpus)
 - More **compute** (training time)
- **Emergent Capabilities**: Abilities that appear suddenly at scale.
 - GPT-2 (1.5B): Basic text generation.
 - GPT-3 (175B): Few-shot learning, basic reasoning.
 - GPT-4 (1T+?): Complex code, multi-step problem solving.

Implication

You don't need to understand *why* scaling works. You need to know *that* it works.

Context Window & Memory

- LLMs have a **Context Window** (e.g., GPT-4o: up to 128k tokens).
- **No true memory**: The LLM only "knows" what's in the current prompt.
- **Implication**: Long conversations or large codebases can exceed the window.

What This Means for You

- Feed relevant context explicitly.
- Don't expect the AI to "remember" a chat from yesterday.
- Tools like RAG inject external documents into the context.

Why Are LLMs So Powerful?

1. **Scale:** Billions of parameters, trillions of training tokens.
2. **Versatility:** Zero-shot, few-shot, and in-context learning.
3. **Emergent Capabilities:** Reasoning, code generation, translation, explanation.
4. **Ecosystem:** Tools, agents, plugins, APIs built on top.

They are statistical pattern matchers operating at an unprecedented scale.

Limitations of LLMs

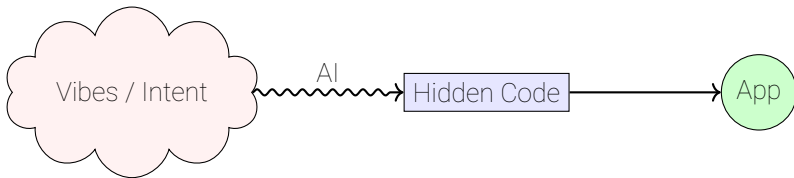
- **Hallucination**: Confidently incorrect output. Invents facts, libraries, APIs.
- **No Real-World Grounding**: Cannot browse the web (unless given tools), no persistent knowledge after training cutoff.
- **Prompt Sensitivity**: Small changes in wording can lead to vastly different outputs.
- **Compute Cost**: Energy-intensive, latency can be high, expensive to run.

Key Takeaway

LLMs are powerful, but they are **not reliable sources of truth**. Always verify.

Mode 1: "Vibe Coding"

- **Definition:** An emerging approach where developers interact with AI primarily through natural language and "vibes".
- Coined by Andrej Karpathy (Feb 2025): *"Embrace the vibes, forget the code."*
- **Philosophy:** Functionality > Implementation details.



Adoption & Impact

- **Startups:** "25% of YC Winter 2025 startups have codebases that are 95% AI-generated." (TechCrunch)
- **Jobs:** AI fluency is now required in 85% of tech startup job listings.
- **Shift:** Skills are moving from "Writing Syntax" to "Reviewing Logic" and "System Design".

The "Vibe" Spectrum

1. Vibe Coding

- Quick prompt
- "Just make it work"
- **Good for:** Prototypes, Scripts

2. Vibe Engineering

- Structured prompts
- Modular thinking
- **Good for:** MVPs, Features

3. Context Engineering

- Constraint-based
- Verification loops
- **Good for:** Production

We want to move towards the Right (Context Engineering) for reliable software.

Pros of Vibe Coding

- **Rapid Development:** Build prototypes in hours, not days.
- **Enables Non-Coders:** Founders, designers, and domain experts can create apps.
- **Increased Productivity:** Delegate boilerplate and routine tasks to AI.
- **Disposable Software:** Quick scripts, one-off tools, internal utilities.
- **Highly Personalized Apps:** Build something for *your specific need*.

Cons of Vibe Coding (The Dangers)

Critical Risks

- **Lack of Understanding**: You can't debug what you don't understand.
- **Accountability**: Who is responsible when AI-generated code fails?
- **Buggy & Insecure**: AI code often has subtle bugs and security holes.
- **Unmaintainable**: Code quickly becomes a "black box" that no one can extend.
- **Not Suitable for Production**: Fine for prototypes, dangerous for live systems.

We must improve on Vibe Coding to build reliable software!

Expert Perspectives

Simon Willison: "Risky for production systems due to shallow understanding. **Code MUST be checked.**"

Andrew Ng: "Warns against trivializing engineering with buzzwords." (AI needs rigor).

Amazon (Deepak Singh): "AI is a coding *collaborator*, not a replacement."

Key takeaway: AI augments engineers; it does not replace the need for expertise.

Best Use Cases for Vibe Coding

When Vibe Coding Shines

- Weekend projects and hackathons.
- Personal utilities and internal scripts.
- MVPs and proof-of-concept demos.
- Educational prototypes and learning exercises.
- Experimental workflows where failure is cheap.

When to Avoid

- Production systems with real users.
- Regulated industries (healthcare, finance, law).
- Any code that must be maintained long-term.

Mode 2: Context Engineering

- **Definition:** Deliberately structuring inputs (context) to guarantee reliable, scalable, and safe AI outputs.
- It treats the Prompt as a **Function Call** with strict parameters.



The Context Stack

To get good results, you must engineer the full stack:

Task: "Refactor Login.java to use AuthService"

Project: File Structure, Existing Classes, Utils

Rules: "Use JUnit 5, CheckStyle, No Deprecated APIs"

System Prompt: "You are a Principal Java Architect"

Technique 1: Role Conditioning

Generic Prompt

- "Write code to sort a list."

Result: simple Bubble Sort or 'Collections.sort'.

Conditioned Prompt

- "Act as a **Performance Engineer**. Write a sort for a 1TB dataset with limited RAM."

Result: External Merge Sort with memory mapping.

Technique 2: Few-Shot Prompting

Zero-Shot: Asking the model to do something without examples.

Few-Shot: Providing 1-3 examples of Input → Output.

Example: Writing Unit Tests

Context: Input: 'add(int a, int b)' → Output: '@Test void testAdd() assert(add(1,1)==2);'

Input: 'div(int a, int b)' → Output: ... *(Model follows the pattern)*

This drastically improves consistency with your project's coding style.

Technique 3: Chain of Thought (CoT)

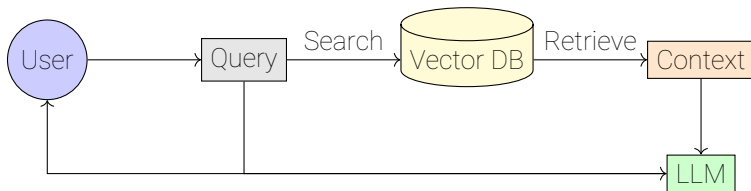
- LLMs often fail at complex logic if forced to answer immediately.
- **Technique:** Ask the model to "Think step by step".
- Determining the logic *before* writing the code reduces bugs.

Prompt: `"Explain your logic first, then write the Java code."`

Technique 4: RAG (Retrieval Augmented Generation)

Problem: LLMs don't know your private code or docs.

Solution: Inject relevant snippets dynamically.



AI in the SDLC

AI Assists across the entire Software Development Life Cycle:

1. **Requirements:** "Clarify this Ticket" (CoT)
2. **Design:** "Propose 3 Architectures" (Role Conditioning)
3. **Implementation:** "Write the Skeleton" (Vibe Coding)
4. **Testing:** "Generate JUnit cases" (Few-Shot)
5. **Review:** "Check for security flaws"

Key Components of Context

System Prompts: Set tone and behavior (“You are a helpful Java expert”).

Examples (Few-Shot): Demonstrate desired input-output patterns.

Metadata: Inject user info, task settings, domain constraints.

Memory/State: Store prior messages or structured data (simulated via prompts).

These components form the “context stack” that the LLM uses to generate responses.

Guardrails: Preventing Misbehavior

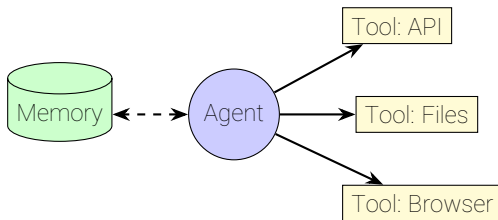
- **Definition:** Rules or code that constrain AI outputs.
- **Preambles:** System prompts like “Never suggest a diagnosis” or “Always cite sources”.
- **Post-Processing:** Filter outputs for sensitive data (PII, secrets).
- **Validation:** Check AI output against schemas or rules before using.

Example

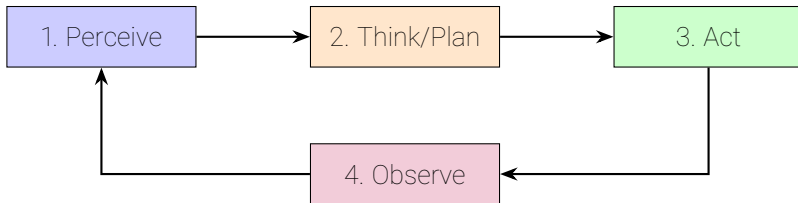
A healthcare AI might have a guardrail: “If the user asks for medication advice, respond with: ‘Please consult a doctor.’ ”

What are AI Agents?

- **Definition:** AI systems that can *autonomously* take actions to achieve goals.
- **Beyond chat:** Agents don't just respond---they *plan, execute, and iterate*.
- **Key capabilities:**
 - Use external **tools** (APIs, file systems, browsers)
 - Maintain **memory** across interactions
 - **Self-correct** based on feedback



The Agent Loop



- **Perceive:** Read context, user input, tool outputs.
- **Think/Plan:** Decide what to do next (LLM reasoning).
- **Act:** Call tools, write code, execute commands.
- **Observe:** Check results, iterate until goal is met.

Examples of AI Agents

- **Coding Agents:** GitHub Copilot Agent, Cursor, Devin, Claude Code
 - Read codebase, write code, run tests, fix errors autonomously
- **Research Agents:** Perplexity, ChatGPT Deep Research
 - Browse web, synthesize sources, produce reports
- **Computer Use Agents:** Claude Computer Use, Operator
 - Control mouse/keyboard, interact with any application

The Trend

We are moving from **chat assistants** → **autonomous agents**.

Prompting vs Context Engineering vs Agentic Systems

Aspect	Prompting	Context Engineering	Agentic Systems
Input	Single prompt	Prompt + metadata + examples	Context + memory + tools
Goal	One response	Consistent outputs	Autonomous task execution
Structure	Unstructured	Highly structured	Orchestrated pipeline
Reproducibility	Low	High	Very High

Context Engineering is the bridge between simple prompting and full AI agents.

The File-Based Workflow: `Rules.md`

Global project rules injected into every AI interaction:

- **Project awareness:** Read planning docs, follow tasks.
- **Code structure:** File limits, module layout.
- **Testing:** Unit test patterns, coverage rules.
- **Style guide:** Language preferences, formatting (Google Style?).
- **Documentation:** Docstrings, commenting style.

Acts as a **global context** that keeps the AI consistent across all prompts.

The Feature Spec: `FEATURE.md`

Defines scope and requirements for a single feature:

- **Feature:** Clear statement of what to build.
- **Examples:** Provide reference files or I/O samples.
- **Documentation:** Links to APIs or specs.
- **Considerations:** Edge cases, security, known gotchas.

Provides AI with **task-specific context** to shape its output.

Product Requirements Prompt (PRP)

Transforms requests into AI-ready implementation plans:

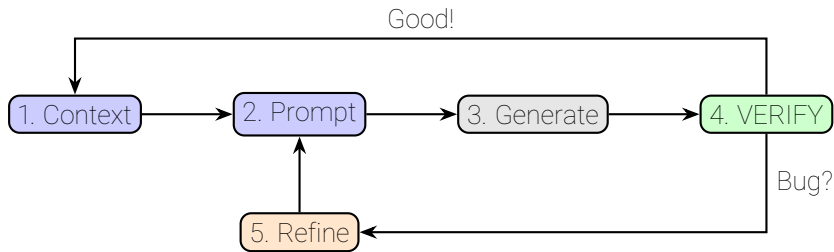
1. **Goal / Why:** Business intent and reasoning.
2. **What:** Feature behavior, specs, UX/API flow.
3. **Context:** Docs, examples, code patterns, data schemas.
4. **Implementation Blueprint:** Step-by-step design + validation.

Ensures AI has a **complete understanding** of what, why, and how.

SDLC Context Table

Phase	Context Input	AI Task
Requirements	Feature description, domain facts	Write user stories, clarify scope
Design	UML, patterns, constraints	Propose components, data flows
Implementation	Style guide, task info, examples	Write or complete code
Testing	Function + known bugs	Generate unit tests
Review	Diff + policies + memory	Suggest comments, rate changes
Deployment	Platform, IaC templates	Write configs, CI/CD pipelines

The Verified Loop



Rule of Thumb

For every minute you save typing code, spend 30 seconds verifying it.

Risks & Mitigation

- **Hallucination:** The AI invents a library function that doesn't exist.
 - *Fix:* Check docs. Compile often.
- **Security:** Pasting secrets (API Keys) into the chat.
 - *Fix:* NEVER do this. Use placeholders.
- **Skill Rot:** Forgetting the basics.
 - *Fix:* Code manually sometimes. Understand, don't just copy.

Paradigm: Full Project vs. Feature-by-Feature

Full Project Scaffolding

- "Build me a Todo app in Spring Boot."
- Good for: Initial setup, greenfield.
- Risk: Large diffs, hard to review.

Feature-by-Feature

- "Add user authentication to this app."
- Good for: Existing codebases, iteration.
- Risk: Context loss between features.

Best Practice

Start with scaffolding, then switch to feature-by-feature as the codebase grows.

Handling Failures Early (Testing First)

- **Compile Often:** Don't let errors accumulate. Fix early.
- **Test Early:** Write tests *as you go*, not at the end.
- **Small Commits:** Review AI output in small chunks.
- **Fail Fast:** If AI gets stuck, restart the context or try a different prompt.

The Compound Error Problem

Errors compound. One bad AI-generated function leads to more bad code on top. **Validate each step before moving on.**

The Danger of Not Understanding the Code

- **You Can't Debug What You Don't Understand.**
 - AI code may work, but *why* does it work?
 - When it breaks, you're stuck.
- **Skill Rot:** Relying too heavily on AI leads to forgetting fundamentals.
- **Long-Term Maintenance:** Code you didn't write is code you can't easily change.

The Golden Rule

Never merge code you couldn't have written yourself (given more time).

Summary

- AI is a powerful **Abstraction Layer**, effectively a new programming language (English).
- **Vibe Coding** allows for rapid creation, but lacks rigorous control.
- **Context Engineering** provides the structure needed for professional software.
- Understanding the **Transformer** and **Probabilistic** nature of LLMs helps you prompt better.